Lecture 2: Stacks and Queues

Stack → push(element), pop(), element& top(), size(), empty()

Array / Vector: base_ptr → [1|2|3|], top_ptr one past last, capacity 4	**Linked List (Singly)**: head_ptr → A → B → C → nullptr, size 3

Time complexity: $O(1)$ for all functions for both implementations, array better

std::stack default: std::deque, can use vector or list by:

```
(1)  std::stack<int, std::list<int>> myStack;
```

Queue → push(element), pop(), element& front(), size(), empty()

Circular Buffer Array: [A|B|], front_idx at A, back_idx one past B, capacity 3, size 2	**Linked List (Singly)**: head_ptr → A → B → C → nullptr, tail_ptr at C, size 3

Time complexity: $O(1)$ for all functions but array better

std::queue default: std::deque, can use list (no vector as queue::pop = pop_front) by:

```
(2)  std::queue<int, std::list<int>> myQueue;
```

Note → Stacks and queues do not allow iterators, deques do

Deque → push_front(), pop_front(), front(), push_back(), pop_back(), back(), size(), empty()

Circular Buffer Array: front_idx → [A|B|C|D] ← back_idx	**Linked List (Doubly)**: head_ptr → A ⇄ B ⇄ C ⇄ D ← tail_ptr

Time complexity: $O(1)$ for all functions but array better

Priority Queue → push(element), pop(), const element& top(), size(), empty()

std::priority_queue: max PQ (higher value = higher priority), uses std::less

min_PQ: implemented via the following

```
(3)  std::priority_queue<T, vector<T>, std::greater<T>>
```

Underlying impl.	**Insert**	**Top**	**Remove**
Unordered sequence	Const	Linear	Top+Const
Sorted sequence	Linear	Const	Const
Heap	Log	Const	Log
Array of linked lists (small int priorities)	Const	Const	Const

Note: remove() in unordered removes highest priority and replaces w/ back element

Manual PQ using <algorithm>: std::make_heap(), push_heap(), pop_heap()

Container gotchas:

- **Circular buffer:** full = after push, back == front. Empty = after pop, front == back. Reaching array end = wrap-around, not full.
- **LL vs array stack:** LL push is $\Theta(1)$, so slow push is *not* a downside. Real downsides: per-node allocation, memory overhead, size() $\Theta(n)$ if untracked.
- **ID container from output:** LIFO → stack; FIFO → queue; largest first → priority_queue.
- Interleaving two vectors/queues/stacks: $\Theta(n)$ each (stacks reverse via temp stack).
- Stream where each value is new min or new max → deque, $\Theta(n)$ total.
- FIFO inventory → queue; LIFO inventory → stack.

Lecture 3: Complexity Analysis

Meaning: If $f(n)$ = # of steps, how does $f(n)$ grow?

Factors that change $f(n)$: n > algorithm, implementation, CPU, parallel programs

What counts as step: variable assignment, arithmetic op, comparison, array indexing / pointer reference, function call + return (not counting data taken as argument / return value)

$O(1)$	Constant	$O(n^2)$	Quadratic
$O(\log n)$	Logarithmic	$O(n^3), O(n^4)$	Polynomial
$O(n)$	Linear	$O(c^n)$	Exponential
$O(n \log n)$	Loglinear	$O(n!)$	Factorial
		$O(2^{2^n})$	Doubly Exp.

For Loop: $2n+2$

Big-O Notation: $f(n) = O(g(n))$ if for some constant c, $c \cdot g(n) \geq f(n)$ eventually OR (sufficient but not necessary) $\lim_{n\to\infty} f(n)/g(n) = d < \infty$

Loop patterns: `i*=2` / `n/=2` → $\log n$; inner `j<i` (triangular) → $\sum i = n^2/2 = \Theta(n^2)$; step `k+=`$\sqrt{n}$ up to n → $\sqrt{n}$; `i` from $n/2$ to n → $\Theta(n)$ (constants drop); independent nested loops → multiply; sequential loops → add, keep largest.

Growth order: $1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < n^3 \log n < 2^n < 3^n < n! < n^n$

Log facts: $\log(n!) = \Theta(n \log n)$; log bases differ by a constant; $\log(n^a b^n) = \Theta(n)$; $\log(n^4) = 4 \log n$ (`bar(n^4)` costs $4 \log n$).

Asymptotic truths:

- "$\Theta(n \log n)$ *always* faster than $\Theta(n^2)$" is **false**: constants/small n flip it.
- Oscillating f can be neither $O(g)$ nor $\Omega(g)$ (order is partial).
- $f+g = O(h) \Rightarrow$ both $f, g = O(h)$. $f = O(g) \Leftrightarrow g = \Omega(f)$.
- Halving and thirding (binary/ternary split) both $\Theta(\log n)$.

Sort then search? Sort when #searches $k = \Omega(\log n)$, since $kn \geq n \log n + k \log n$.

Amortized: total cost of n ops ÷ n. Vector doubling: push_back $\Theta(1)$ amortized. Growing by constant (+k): $\Theta(n)$ amortized. Expensive op only at powers of 2 or 3: geometric sum = $\Theta(n)$ total → $\Theta(1)$. Resize that doubles **and sorts**: $\Theta(\log n)$ amortized.

Lecture 4: Measuring Runtime Performance

Ways to Measure → Analytically = look at code + Empirically = wall-clock runtime

Using Timer: Create with Timer t, has functions start(), stop(), reset(), seconds()

Runtime Analysis Tools → ./progName -options args = 0.84 user (time spent by your program), 0.00 system (CPU time spent on other work to get your program functional), 0:00:85 elapsed (clock time), 99% CPU ((user + system) / elapsed)

Memory Leak → valgrind ./program b 100000 (use `--leak-check=full` and `--show-leak-kinds=all`)

Empirical ≠ analysis: input too small, wrong algorithm ran, or bad analysis. Worst-case input makes it *slower*, never faster.

Master Theorem & Lecture 5: Recursion

Master Theorem → $T(n) = a \cdot T(n/b) + f(n)$ where $a \geq 1$, $b > 1$ and const., $f(n)$ is polynomial + monotonically increasing, c = highest degree of f

$a > b^c$	$\Theta(n^{\log_b a})$
$a = b^c$	$\Theta(n^c \log n)$
$a < b^c$	$\Theta(n^c)$

$T(n/2) + c$	Binary Search	$O(\log n)$
$T(n-1) + c$	Linear Search	$O(n)$
$2T(n/2) + c$	Tree Traversal	$O(n)$
$T(n-1) + c_1 n + c_2$	Selection Sorts	$O(n^2)$
$2T(n/2) + c_1 n + c_2$	Merge/Quick	$O(n \log n)$

k subproblems: if a recurrence divides a problem into k subproblems and one is processed → $T(n) = \Theta(\log_k n)$; all are processed → $\Theta(n)$; $m < k$ are processed → $\Theta(n^{\log_k m})$

Not applicable: a not constant ($a = n$); two different subproblem sizes; subtracts ($T(n-1)$) instead of divides.

Subtract recurrences: $T(n-1) + n = \Theta(n^2)$; $2T(n-1) + c = \Theta(2^n)$; $aT(n-1) + c = \Theta(a^n)$.

From code: recursive call inside a loop of n iterations → $n \cdot T(\dots)$. Count nested-loop helper calls: $n^2 \cdot n$ calls at $\log n$ each = $n^3 \log n$.

Shortcut: $T(n) = T(n/b) + \Theta(n)$ is always $\Theta(n)$.

Program Stack:

- Function call made → local variables + argument values pushed onto program stack
- Function call received (begins executing) → arguments popped off stack (as they need to be used)
- Return reached → return value pushed onto program stack
- Return received at call site (execution goes back to caller) → return value popped + saved local variables restored
- Program stack supports nested function calls + only one program stack (per thread) + program stack size is limited
- Recursion memory: stack depth counts as space. Non-tail Θ(depth); tail can be $\Theta(1)$.

Lecture 6: Arrays and Containers

1D → 2D: column = index % num_columns; row = index / num_columns

2D → 1D: index = row * num_columns + column

Fixed Array: Stored on stack, a[i][j] is one memory operation, size fixed at compile time

Dynamic Array: Stored on heap, supports triangular arrays, copying + swapping rows is quick, a[i][j] is 2 memory operations, size changeable at runtime

Two Methods: 1. Go through rows and columns, and access arr[r][c]; 2. Static initialization where each row vector in {} and entire thing in {}	(4) `int** a = new int*[rows];` Loop through rows (5) `a[r] = new int[cols];` Loop through rows (6) `delete[] a[r];` (7) `delete[] a;`

Containers → Access: Sequential vs Random Access

What to Store	**Value**	**Pointer**	**Reference**
Ex.	`char data;`	`char *data;`	`char &data(c);`
Owner	Only container edits/deletes	Container or other object	None: cannot delete by reference
Drawback	Large objects take time to copy	Unsafe	Must be initialized but cannot be assigned to
Usage	Most common	Used for char*, shared data	Impractical in most cases

What to Get	**Value**	**Ptr, Const ptr**	**Reference, const ref**
Ex.	`char getElt(int);`	`char *getElt(int);`	`char &getElt(int);`
Notes	Costly for copying large objects	Unsafe, pointer may be invalid	Usually a good choice

Lecture 7: STL

Keywords → explicit = prevents implicit type conversion, mutable = member variable can be modified by const member function

Containers and Iterators → .begin(), .end(), .cbegin(), .cend(), .rbegin(), .rend()

Container	**Pointers**	**.size()**
list<>	Doubly-linked	$O(1)$
slist<>	Singly-linked	Can be $O(n)$
forward_list<>	Singly-linked	Does not exist

Iterator ops	**In**	**Out**	**Fwd**	**Bidir**	**Rand**
Deref (*) and read	✓		✓	✓	✓
Deref (*) and write		✓	✓	✓	✓
Forward movement (++)	✓	✓	✓	✓	✓
Backward movement (--)				✓	✓
Multiple passes			✓	✓	✓
== and !=	✓		✓	✓	✓
Pointer arith (+, -)					✓
Pointer compare (<, >)					✓

Memory Overhead: vector < list < unordered_map

Vector: Internally has 3 pointers, 3d vector overhead = $3 + 3a + 3ab$, organize $a < b < c$

Vector size vs capacity:

- `reserve(x)`: capacity only, size unchanged; smaller x does nothing.
- `resize(x)`: changes size, zero-fills new slots; capacity grows only if $x >$ capacity; shrinking keeps capacity.
- `resize(n)` then `push_back` appends **after** n zeros → size $2n$.
- `reserve(n)` then `[]` writes past `size()` → **undefined behaviour**.
- Out of memory fix: `resize` then `[]`, or `reserve` then `push_back`.
- `erase` never shrinks capacity. Capacity 1 → 2 → 4 → 8 → 16; after 10 push_backs capacity is 16.
- `for (int x : v)` copies; changes don't stick. Use `auto&` to modify.
- `a[i] == i[a] == *(a+i)`. `enum class` fits a fixed set of named categories.

Functors →

```
(8)  bool operator()(const Employee& left, const
       Employee& right) const;
```

Randomization → Ex:

```
(9)  int x = static_cast<int>(mt() % 21) - 10; //
       produces [-10,10]
```

<random>: Randomization provided by STL, great for testing → best to use <random> library (better than older libraries like rand(), etc.), has 2 parts: 1) Generators: produce random #s (std::random_device, std::mt19937, std::mt19937_64); 2) Distribution: turns #s into desired range (uniform_int_distribution, uniform_real_distribution, normal_distribution). Can also achieve similar result by iota() + shuffle() on a container (e.g. vector)

STL Performance → compare vector, vector_pre (vector preallocated size so no $O(n)$ grow), list, deque:

- push_back(): vector_pre > vector > deque > list → if you know size ahead of time, vector.reserve() (preallocation) is the way to go
- find(): vector > deque > list → vector stores elements next to each other so CPU cache works well; all are $O(n)$

- random_insert(): list > deque > vector → list = changing pointers, deque = can shift elements from closer end, vector = move all elements
- random_removal(): list > deque > vector
- fill_front(): deque / list > vector → deque and list are $O(1)$, vector very slow

Lecture 8: Heaps, Priority Queues, Heapsort

Trees → connected graph w/o cycles / graph where any 2 nodes connected by unique path

Directed tree: Child / parent relationships. Height, Size, Depth:

```
(10) height(empty) = 0; height(node) =
       max(height(leftChild), height(rightChild)) + 1
(11) size(empty) = 0; size(node) = size(leftChild) +
       size(rightChild) + 1
(12) depth(empty) = 0; depth(node) = depth(parent) + 1
```

Complete Binary Tree (1 indexing) → parent index = $i/2$, left child = $2i$, right child = $2i+1$, index is leaf if $2i > n$

0 indexing (exam arrays): parent = $(i-1)/2$; children $2i+1$, $2i+2$; last internal node = $n/2-1$

Heap-Ordered: Tree is (max) heap ordered if each node's priority ≤ priority of parent

Heap: DS w/ easy access to most extreme element (must be complete + heap-ordered)

Valid heap? (1) complete: no gaps, filled left to right; (2) every parent ≥ children (max) or ≤ (min); (3) duplicates fine; (4) root is the extreme.

FixUp →

```
(13) void fixUp(Item heap[], size_t k) {
(14)   while (k > 1 && heap[k/2] < heap[k]) {
(15)     swap(heap[k], heap[k/2]); k /= 2;
```

FixDown →

```
(16) void fixDown(Item heap[], size_t heapsize, size_t k)
       {
(17)   while (2 * k <= heapsize) {
(18)     size_t j = 2 * k;
(19)     if (j < heapsize && heap[j] < heap[j+1]) ++j;
(20)     if (heap[k] >= heap[j]) break;
(21)     swap(heap[k], heap[j]); k = j;
```

Priority Queue → Implemented w/ heaps as insertion / removal = same time complexity

Insertion / Deletion:

```
void push(Item n) {heap[++heapsize] = n; fixUp(heap,
  heapsize);}
void pop() {heap[1] = heap[heapsize--];
  fixDown(heap, heapsize, 1);}
```

Time Complexities	Insert	Inspect	Remove
Unordered array PQ	$O(1)$	$O(n)$	$O(n)$
Sorted array PQ	$O(n)$	$O(1)$	$O(1)$
Heap	$O(\log n)$	$O(1)$	$O(\log n)$

Push trace: append at end, fixUp (swap with parent while out of order).

Pop trace: last element to root, fixDown: swap with **larger** child (max-heap) / **smaller** child (min-heap).

Update key: max-heap: increase → fixUp, decrease → fixDown. Min-heap: reverse.

Heapify → Build heap from existing DS: 1. Add elements one at time = $O(n \log n)$; 2. Sort array = $O(n \log n)$ (reverse order for max heap); 3. Heapify DS: bottom-top + fixDown = $O(n)$, top-bottom + fixUp = $O(n \log n)$

Bottom-up heapify trace: fixDown each internal node from last internal node back to root. $\Theta(n)$ time, $\Theta(1)$ memory.

Heapsort → Converts unsorted array into sorted array in ascending order using a heap

```
(22) void heapsort(Item heap[], size_t n) { // O(n logn)
(23)   heapify(heap, n);
(24)   for (size_t i = n; i >= 2; --i) {
(25)     swap(heap[i], heap[1]); fixDown(heap, i-1, 1);
```

Costs: pushing n one at a time $\Theta(n \log n)$; PQ range constructor $\Theta(n)$. Heapsort $\Theta(n \log n)$ even if heapify were $O(1)$ (n pops dominate); unstable, in place, non-adaptive.

PQ comparator rule: `comp(a,b)==true` means a has **lower** priority, sits below b. `less` → max-heap; `greater` → min-heap. Lambda comparators need `decltype(cmp)` and must be passed to the constructor.

Heap pattern	Approach / cost
k largest items	min-heap size k; $\Theta(n \log k)$ time, $\Theta(k)$ space
kth largest in $O(1)$	min-PQ size $k-1$ + max-PQ for rest
each item ≤ d from sorted spot	min-heap size $d+1$; $\Theta(n \log d)$
merge k sorted lists	min-PQ of list heads; $\Theta(N \log k)$
median	max-heap lower half + min-heap upper half

Lecture 9: Ordered and Sorted Ranges, Algorithms and DS to Represent Sets

Binary Search → $O(\log n)$, best is $O(1)$, worst is $\Theta(\log n)$

Algorithm: Two iterators right and left, and until they do not cross, if our midpoint is less than value to find, left iterator becomes one beyond midpoint, otherwise the right becomes the midpoint

Note: binary_search() returns a bool, not the location → to find location, use functions that return iterators: lower_bound() = first iterator not less than target; upper_bound() = first iterator greater than target (both: end iterator if none found); equal_range() returns pair (lb, ub), all items equal to target. Ex: equal_range(begin(v), end(v), 73) on [2,3,5,73,73,73,85,91,92] → lb at first 73, ub at 85.

Binary search tricks: works on *any* monotonic condition. Rotated sorted array: compare mid with hi. Single unpaired element: pairs start at even indices before it, odd after. Sorted 2D matrix: start top-right, $\Theta(m+n)$.

Sets → union (∪), intersection (∩), set difference (\), symmetric diff (÷), addElement (+), isElement (∈), isEmpty

setUnion: Definition + Usage

```
(26) template<class InIterator1, class InIterator2, class
       OutIterator, class Compare>
(27) OutIterator set_union(InIterator1 first1,
       InIterator1 last1, InIterator2 first2, InIterator2
       last2, OutIterator result, Compare compare) {
(28)   while (first1 != last1 && first2 != last2) {
(29)     if (compare(*first1, *first2)) *result++ =
           *first1++;
(30)     else if (compare(*first2, *first1)) *result++ =
           *first2++;
(31)     else {*result++ = *first1++; ++first2;}
(32)   while (first1 != last1) {*result++ = *first1++;}
(33)   while (first2 != last2) {*result++ = *first2++;}
     // Method 1
(34) vector<string> out; out.resize(v1.size() +
       l1.size());
(35) auto it = set_union(begin(v1), end(v1),
       begin(l1), end(l1), begin(out), less<string>());
       out.erase(it, end(out));
     // Method 2
(36) set_union(begin(v1), end(v1), begin(l1), end(l1),
       back_inserter(out), less<string>());
```

Time Complexity: Assuming set is implemented with sorted array / container

initialize()	$O(1)$ or $O(n \log n)$
clear()	$O(1)$ or $O(n)$
isMember()	$O(\log n)$
copy()	$O(n)$
set_union()	$O(n)$
set_intersection()	$O(n)$

initialize: $O(1)$ = elements already sorted, so out container just points to them; $O(n \log n)$ = elements not sorted but set impl. requires sorted, so you sort.

clear: $O(1)$ = set represented by range [first, last), doing first = last clears set; $O(n)$ = physically delete all n elements.

Other ops: union/intersection/difference/symmetric diff $\Theta(n+m)$; isElement $\Theta(\log n)$; addElement $\Theta(n)$.

- `set_*` need sorted input, return iterator to output end. Use `back_inserter` to avoid presizing.
- `unique` removes only **adjacent** duplicates: sort first, then `erase(unique(...), end)`.
- `minmax_element` returns **first** min and **last** max.

unionFind: find() – check if an element belongs to a particular set (or if 2 elements belong to same set) ($O(\alpha(n)) \sim O(1)$); union() – merge 2 sets together into single set ($O(\alpha(n)) \sim O(1)$)

Implementation: Elements divided into n containers, each element has a representative, every set also has an ultimate representative (element that is its own representative).

- During find(), path compression changes each element in that find() call's representative to ultimate representative
- During union(x,y), find repre. of x, repre. of y, make one repre point to the other
- #sets = #roots (items whose rep is themselves). Min unions to merge all = #sets −1.
- find(x): follow chain to root; full compression points **every node on that path** directly at root. Changes nothing only if x already points straight at root.
- Union by size/rank: attach smaller tree under larger.

Lecture 10: Elementary Sorting Methods

Overview → Elementary sorts = bubble, selection, insertion, counting, arrays and linked lists can be sorted (we do arrays), you want a sort to use $O(n)$ memory overhead or less, stable sorting = preservation of order of items w/ duplicate keys, adaptive sort = different operations on diff. input

Size Affects Sorting → Internal sort: copying elements cheap + all data fits in memory = sort data. Indirect sort: copying elements expensive + all data fits in memory = sort indices of data. External sort: data doesn't fit in memory = access sequentially / pull blocks into RAM and sort there

Bubble Sort → stable; adaptive if early-exit on no-swap pass (can be both)

Algorithm (Adaptive): Left and right iterators, left goes until end, right goes until it crosses left, if value right is less than value to its left, swap. If no swaps in entire pass then end

Complexity: Non-adaptive = comparisons[$n^2/2$, -, -] + swaps[$n^2/2$, $n^2/2$, -], adaptive = comparisons[$n^2/2$, $n^2/2$, n] + swaps[$n^2/2$, $n^2/2$, $n \to 0$]

Selection Sort → $O(n^2)$, pre-order hardly matters, non-adaptive, unstable

Algorithm:

```
(37) for (auto i = left; i < right - 1; ++i) {
(38)   auto minIndex = i;
(39)   for (auto j = i + 1; j < right; ++j)
(40)     if (a[j] < a[minIndex]) minIndex = j;
(41)   if (minIndex != i) swap(a[i], a[minIndex]); //
         remove if for non-adap
```

Complexity: Non-adaptive = comparisons[$n^2/2$,-,-] + swaps[$n-1$, $n-1$, $n-1$], adaptive = comparisons[$(n^2-n)/2+n-1$] + swaps[$n-1$,-,0]

Insertion Sort → Best sort for small n, stable; best $\Theta(n)$ (sorted), avg/worst $\Theta(n^2)$

Algorithm:

```
(42) for (auto i = right - 1; i > left; --i)
(43)   if (a[i] < a[i-1]) swap(a[i-1], a[i]);
(44) for (auto i = left + 2; i < right; ++i) {
(45)   Item v = a[i]; auto j = i;
(46)   while (v < a[j - 1]) {
(47)     a[j] = a[j-1]; --j; }
(48)   a[j] = v;
```

Complexity: Non-adaptive = comparisons[$n^2/2$,-,-] + swaps[$n^2/2$,$n^2/2$,-], adaptive = comparisons[$n^2/2$,$n^2/4$,n] + copies[$n^2/2$, $n^2/4$, n]

Rules: Insertion + bubble good when every element close to correct pos, insertion good when you add small # of new items to already sorted data, selection good when swapping expensive

Counting Sort → stable, good for sorting a limited # of different keys, $O(n+k)$ (time and space)

Algorithm: Count how many times each key appears, calculate offsets, figure out where each key belongs in sorted group, copy items into output in their correct groups

std::sort → Expects two iterators, begin and end + operator< for data type. Functor: ascending sort – return true when a < b and vice versa

Sort	Best	Avg	Worst	Stb	Adp	Mem
Bubble*	n	n^2	n^2	✓	✓	$O(1)$
Selection	n^2	n^2	n^2	×	×	$O(1)$
Insertion	n	n^2	n^2	✓	✓	$O(1)$
Counting	$n+k$	$n+k$	$n+k$	✓	×	$O(n+k)$
Heapsort	$n \log n$	$n \log n$	$n \log n$	×	×	$O(1)$
std::sort	$n \log n$	$n \log n$	$n \log n$	×	–	$O(\log n)$
stable_sort		$n \log n$		✓		$O(n)$

- *Bubble with early exit. Selection: n^2 comparisons even on sorted input, but only $n-1$ swaps.
- Non-adaptive advantage: simpler, predictable. Not faster, not guaranteed stable.
- Binary insertion sort can stay stable.

Coding Patterns (recognition)

Signal	Pattern
prev/next greater, "days until warmer"	monotonic stack (or reuse output array as jump pointers for $O(1)$ space)
pair sum closest to k, shift zeros, dedupe sorted	two pointers / read–write index
runs of k identical values collapse	stack of (value, count)
reverse order of chunks between separators	reverse whole, then reverse each chunk
add MSD-first numbers in lists	push digits on stacks, prepend result nodes
linked-list building	dummy head + tail pointer
reverse linked list	prev / cur / next pointers
browser back/forward	vector + current index; truncate forward history on visit
smallest window to sort	forward scan running max, backward scan running min